ConfigMap & Secret — Lab

Env Vars · Volume Mounts · Combined Pattern

50 minutes | 100 points | 7 tasks | CKA Level

Lab Rules

- ✓ Use kubectl and YAML files only
- ✓ Default namespace unless stated otherwise
- ✓ Apply the ConfigMap and Secret BEFORE creating any Pod
- ✓ Verify every task with kubectl exec or kubectl logs

Part 1 — ConfigMap

50 pts

1 Create a ConfigMap — Imperative

Easy ☆

Create a ConfigMap named **app-config** using the imperative method — no YAML file needed.

Field	Value
ConfigMap Name	`app-config`
Key 1	`APP_ENV = production`
Key 2	`APP_PORT = 8080`
Key 3	`APP_COLOR = blue`
Key 4	`MAX_CONNECTIONS = 100`

Hint:
kubectl create configmap <name> --from-literal=KEY=value --from-literal=KEY2=value2
You can chain multiple --from-literal flags in one command.

Requirements:

a Create the ConfigMap using `--from-literal` for all 4 keys

b Run `kubectl describe configmap app-config` and confirm all 4 keys appear

c Run `kubectl get configmap app-config -o yaml` — what does the output look like?

d What is the difference between a ConfigMap and a regular variable you hardcode in a Pod?

Verify with:

```
kubectl get configmap
kubectl describe configmap app-config
kubectl get configmap app-config -o yaml
```

What is the difference between ConfigMap and a hardcoded variable?

2 Inject ConfigMap as Environment Variables

Easy ☆

Create a Pod that injects **all keys** from the ConfigMap as environment variables using `envFrom`.

Field	Value

File to apply	`02-configmap-as-envvars.yaml`
Pod Name	`env-pod`
Image	`busybox`
Injection method	`envFrom → configMapRef: app-config`
Command	`env && sleep 3600`

Requirements:

a Apply `02-configmap-as-envvars.yaml`

b Run `kubectl logs env-pod` and find the 4 ConfigMap keys in the output

c Run `kubectl exec -it env-pod -- env | grep APP` — what do you see?

d Explain: what does `envFrom` do differently than writing each `env:` key manually?

Verify with:

```
kubectl logs env-pod
kubectl exec -it env-pod -- env | grep APP
kubectl exec -it env-pod -- env | grep MAX
```

What did you see from grep APP?

Difference between envFrom and individual env: keys:

3 Inject Specific Keys with Custom Names

Medium ☆☆

Create a Pod that injects **specific keys** from the ConfigMap with **custom variable names** inside the container.

Field	Value
File to apply	`03-configmap-selective-env.yaml`
Pod Name	`selective-pod`
Image	`busybox`
Env var name	`COLOR ← from APP_COLOR`
Env var name	`PORT ← from APP_PORT`

Note: The variable name inside the container can be different from the ConfigMap key name.

Requirements:

⚙ ConfigMap & Secret — Lab

a Apply `03-configmap-selective-env.yaml`

b Check logs — the pod prints `Color=blue` and `Port=8080`

c Run `kubectl exec -it selective-pod -- env` — is `MAX_CONNECTIONS` visible? Why not?

d When would you use selective injection instead of `envFrom`?

✅ **Verify with:**

```
kubectl logs selective-pod
kubectl exec -it selective-pod -- env
```

✍ Is MAX_CONNECTIONS visible? Why not?

When to use selective injection instead of envFrom:

Part 2 — Secret

30 pts

4 Create a Secret and Inspect It

Easy ☆

Create a Secret named **db-secret** to store database credentials.

Field	Value
Secret Name	`db-secret`
Type	`Opaque (generic)`
Key 1	`DB_USER = admin`
Key 2	`DB_PASSWORD = p@ssw0rd123`
Key 3	`DB_NAME = myapp`

Hint:
Use the imperative method — kubectl handles base64 encoding for you:
kubectl create secret generic <name> --from-literal=KEY=value

If using YAML (05-secret-basic.yaml), values must be pre-encoded:
echo -n "admin" | base64 → YWRtaW4=

Requirements:

a Create the Secret using the imperative method

b Run `kubectl describe secret db-secret` — are the values visible?

c Run `kubectl get secret db-secret -o yaml` — how are the values stored?

d Decode DB_USER: `kubectl get secret db-secret -o jsonpath='{.data.DB_USER}' | base64 -d`

e Explain: why is base64 NOT the same as encryption?

Verify with:

```
kubectl get secret
kubectl describe secret db-secret
kubectl get secret db-secret -o yaml
kubectl get secret db-secret -o jsonpath='{.data.DB_USER}' | base64 -d
```

How are values stored in the Secret YAML?

Why is base64 NOT encryption?

5 Inject Secret as Environment Variables

Medium ☆☆

Create a Pod that injects the **db-secret** values as environment variables and verify they are decoded automatically.

Field	Value
File to apply	`06-secret-as-envvars.yaml`
Pod Name	`db-pod`
Image	`busybox`
Injection method	`envFrom → secretRef: db-secret`
Command	`echo User=$DB_USER && echo DB=$DB_NAME && sleep 3600`

Requirements:

a Apply `06-secret-as-envvars.yaml`

b Run `kubectl logs db-pod` — are the values decoded?

c Run `kubectl exec -it db-pod -- env | grep DB` — can you see DB_PASSWORD?

d Run `kubectl describe pod db-pod` — can you see the Secret values there?

Verify with:

```
kubectl logs db-pod
kubectl exec -it db-pod -- env | grep DB
kubectl describe pod db-pod
```

Are values decoded in logs? What did you see?

Are values visible in kubectl describe pod?

Part 3 — Combined Pattern

20 pts

6 Inject ConfigMap + Secret Together

Medium ☆☆

Create a Pod that uses **both** app-config (ConfigMap) and db-secret (Secret) — injecting both into the same container.

Field	Value
File to apply	`08-combined-cm-and-secret.yaml`
Pod Name	`fullapp-pod`
Image	`busybox`
ConfigMap inject	`envFrom → configMapRef: app-config`
Secret inject	`envFrom → secretRef: db-secret`

Requirements:

a Apply `08-combined-cm-and-secret.yaml`

b Run `kubectl logs fullapp-pod` — verify both sections print correctly

c Run `kubectl exec -it fullapp-pod -- env | sort` — count how many env vars come from ConfigMap vs Secret

d Explain: which vars should go in ConfigMap and which in Secret?

Verify with:

```
kubectl logs fullapp-pod
kubectl exec -it fullapp-pod -- env | sort
kubectl exec -it fullapp-pod -- env | grep -E 'APP|DB'
```

ConfigMap vars (non-sensitive):

Secret vars (sensitive):

Rule: how do you decide which goes where?

7 Deployment with ConfigMap + Secret

Hard ☆☆☆

Apply the Deployment that uses both ConfigMap and Secret. Verify that **all 3 pods** automatically get the same configuration.

⚙ ConfigMap & Secret — Lab

Field	Value
File to apply	`09-deployment-with-config.yaml`
Deployment Name	`webapp-configured`
Replicas	`3`
ConfigMap key used	`APP_ENV`
Secret keys used	`DB_USER, DB_PASSWORD`

Requirements:

a Apply `09-deployment-with-config.yaml`

b Confirm all 3 pods are Running: `kubectl get pods -l app=webapp-configured`

c Pick any pod and verify it has the env vars: `kubectl exec -it <pod> -- env | grep -E 'APP_ENV|DB_USER'`

d Pick a DIFFERENT pod and run the same check — do all pods have the same vars?

e Explain: what is the advantage of using ConfigMap + Secret with a Deployment vs hardcoding values in the YAML?

✅ Verify with:

```
kubectl get pods -l app=webapp-configured
kubectl exec -it <any-pod> -- env | grep -E 'APP_ENV|DB_USER'
kubectl describe pod <any-pod>
```

✍ Did both pods have the same vars? (yes/no)

Advantage of ConfigMap + Secret over hardcoding values: